

Black Friday Data Analysis

In [1]: `# import pandas and numpy`

```
import pandas as pd
```

```
import numpy as np
```

In [2]: `data = pd.read_csv(r"D:\Samson - All Data\Naresh IT Institute\New folder\Black F
data`

Out[2]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
--	---------	------------	--------	-----	------------	---------------	-----------------

0	1000004	P00128942	M	46-50	7	B
---	---------	-----------	---	-------	---	---

1	1000009	P00113442	M	26-35	17	C
---	---------	-----------	---	-------	----	---

2	1000010	P00288442	F	36-45	1	B
---	---------	-----------	---	-------	---	---

3	1000010	P00145342	F	36-45	1	B
---	---------	-----------	---	-------	---	---

4	1000011	P00053842	F	26-35	1	C
---	---------	-----------	---	-------	---	---

...	...	...	...	...	...	...
-----	-----	-----	-----	-----	-----	-----

233594	1006036	P00118942	F	26-35	15	B
--------	---------	-----------	---	-------	----	---

233595	1006036	P00254642	F	26-35	15	B
--------	---------	-----------	---	-------	----	---

233596	1006036	P00031842	F	26-35	15	B
--------	---------	-----------	---	-------	----	---

233597	1006037	P00124742	F	46-50	1	C
--------	---------	-----------	---	-------	---	---

233598	1006039	P00316642	F	46-50	0	B
--------	---------	-----------	---	-------	---	---

233599 rows × 12 columns



In [3]: `type(data)`

Out[3]: `pandas.core.frame.DataFrame`

In [4]: `data.shape`

Out[4]: `(233599, 12)`

In [5]: `data.head()`

Out[5]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_
0	1000004	P00128942	M	46-50	7	B	
1	1000009	P00113442	M	26-35	17	C	
2	1000010	P00288442	F	36-45	1	B	
3	1000010	P00145342	F	36-45	1	B	
4	1000011	P00053842	F	26-35	1	C	

◀ ▶

In [6]: `data.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 233599 entries, 0 to 233598
Data columns (total 12 columns):
#   Column                                Non-Null Count  Dtype
---  -
0   User_ID                               233599 non-null  int64
1   Product_ID                           233599 non-null  object
2   Gender                               233599 non-null  object
3   Age                                   233599 non-null  object
4   Occupation                           233599 non-null  int64
5   City_Category                        233599 non-null  object
6   Stay_In_Current_City_Years          233599 non-null  object
7   Marital_Status                      233599 non-null  int64
8   Product_Category_1                  233599 non-null  int64
9   Product_Category_2                  161255 non-null  float64
10  Product_Category_3                  71037 non-null   float64
11  Purchase                             333 non-null     float64
dtypes: float64(3), int64(4), object(5)
memory usage: 21.4+ MB
```

In [7]: `data.isnull().sum()`

Out[7]:

User_ID	0
Product_ID	0
Gender	0
Age	0
Occupation	0
City_Category	0
Stay_In_Current_City_Years	0
Marital_Status	0
Product_Category_1	0
Product_Category_2	72344
Product_Category_3	162562
Purchase	233266
dtype:	int64

In [8]: `data.isna().sum()`

```
Out[8]: User_ID      0
        Product_ID   0
        Gender       0
        Age          0
        Occupation   0
        City_Category 0
        Stay_In_Current_City_Years 0
        Marital_Status 0
        Product_Category_1 0
        Product_Category_2 72344
        Product_Category_3 162562
        Purchase     233266
        dtype: int64
```

```
In [9]: data = data.fillna(method = 'pad')
```

C:\Windows\Temp\ipykernel_28304\1651801864.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

```
data = data.fillna(method = 'pad')
```

```
In [10]: data.isnull().sum()
```

```
Out[10]: User_ID      0
        Product_ID   0
        Gender       0
        Age          0
        Occupation   0
        City_Category 0
        Stay_In_Current_City_Years 0
        Marital_Status 0
        Product_Category_1 0
        Product_Category_2 0
        Product_Category_3 4
        Purchase     0
        dtype: int64
```

```
In [11]: data[['Product_Category_2', 'Product_Category_3']].head()
```

```
Out[11]:
```

	Product_Category_2	Product_Category_3
0	11.0	NaN
1	5.0	NaN
2	14.0	NaN
3	9.0	NaN
4	5.0	12.0

```
In [12]: data = data.fillna(method = 'backfill')
```

C:\Windows\Temp\ipykernel_28304\1133014479.py:1: FutureWarning: DataFrame.fillna with 'method' is deprecated and will raise in a future version. Use obj.ffill() or obj.bfill() instead.

```
data = data.fillna(method = 'backfill')
```

```
In [13]: data.isnull().sum()
```

```
Out[13]: User_ID          0
        Product_ID      0
        Gender          0
        Age             0
        Occupation      0
        City_Category   0
        Stay_In_Current_City_Years  0
        Marital_Status  0
        Product_Category_1  0
        Product_Category_2  0
        Product_Category_3  0
        Purchase        0
        dtype: int64
```

```
In [14]: #assert that there are no missing values in the dataframe

assert pd.notnull(data).all().all()
```

```
In [15]: # make a copy of dataframe
data1 = data.copy()
```

```
In [16]: # select first row of dataframe

data1.loc[0]
```

```
Out[16]: User_ID          1000004
        Product_ID      P00128942
        Gender          M
        Age             46-50
        Occupation      7
        City_Category   B
        Stay_In_Current_City_Years  2
        Marital_Status  1
        Product_Category_1  1
        Product_Category_2  11.0
        Product_Category_3  12.0
        Purchase        10.0
        Name: 0, dtype: object
```

```
In [17]: #select first row of dataframe

data1.iloc[0]
```

```
Out[17]: User_ID          1000004
        Product_ID      P00128942
        Gender          M
        Age             46-50
        Occupation      7
        City_Category   B
        Stay_In_Current_City_Years  2
        Marital_Status  1
        Product_Category_1  1
        Product_Category_2  11.0
        Product_Category_3  12.0
        Purchase        10.0
        Name: 0, dtype: object
```

```
In [18]: #select first five rows for a specific column
```

```
data1.loc[:, 'Purchase'].head()
```

```
Out[18]: 0    10.0  
        1    15.0  
        2    15.0  
        3    16.0  
        4    19.0  
        Name: Purchase, dtype: float64
```

```
In [19]: #select first row of dataframe  
  
data1.iloc[0]
```

```
Out[19]: User_ID          1000004  
        Product_ID      P00128942  
        Gender          M  
        Age            46-50  
        Occupation      7  
        City_Category   B  
        Stay_In_Current_City_Years  2  
        Marital_Status   1  
        Product_Category_1  1  
        Product_Category_2  11.0  
        Product_Category_3  12.0  
        Purchase        10.0  
        Name: 0, dtype: object
```

```
In [20]: #select last row of dataframe  
  
data1.iloc[-1]
```

```
Out[20]: User_ID          1006039  
        Product_ID      P00316642  
        Gender          F  
        Age            46-50  
        Occupation      0  
        City_Category   B  
        Stay_In_Current_City_Years  4+  
        Marital_Status   1  
        Product_Category_1  4  
        Product_Category_2  5.0  
        Product_Category_3  12.0  
        Purchase        26.0  
        Name: 233598, dtype: object
```

```
In [21]: # get index of first occurrence of maximum Purchase value  
  
data1['Purchase'].idxmax()
```

```
Out[21]: 299
```

```
In [22]: # get value at 1st row and Purchase column pair  
  
data1.at[1, 'Purchase']
```

```
Out[22]: np.float64(15.0)
```

```
In [23]: # get value at 1st row and 11th column pair
```

```
data1.iat[1, 11]
```

```
Out[23]: np.float64(15.0)
```

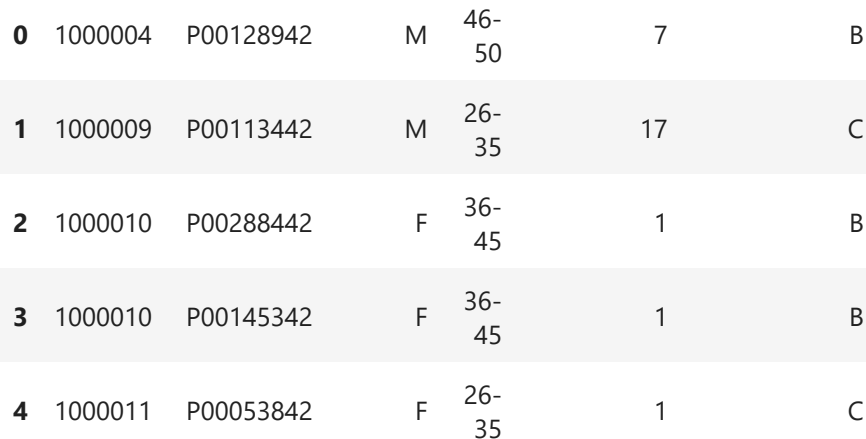
```
In [24]: # make a copy of dataframe df
```

```
data2 = data.copy()
```

```
In [25]: data2.head()
```

```
Out[25]:
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_
0	1000004	P00128942	M	46-50	7	B	
1	1000009	P00113442	M	26-35	17	C	
2	1000010	P00288442	F	36-45	1	B	
3	1000010	P00145342	F	36-45	1	B	
4	1000011	P00053842	F	26-35	1	C	



```
In [26]: # get the purchase amount with a given user_id and product_id
```

```
data2.loc[((data2['User_ID'] == 1000001) & (data2['Product_ID'] == 'P00069042'))]
```

```
Out[26]: Series([], Name: Purchase, dtype: float64)
```

```
In [27]: values=[1000001, 'P00069042', 'F', 0-17, 10, 'A', 2, 0, 3, 6, 14, 8370]
```

```
data2_indexed=data2.isin(values)
```

```
data2_indexed.head(10)
```

Out[27]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_Years
0	False	False	False	False	False	False	
1	False	False	False	False	False	False	
2	False	False	True	False	False	False	
3	False	False	True	False	False	False	
4	False	False	True	False	False	False	
5	False	False	False	False	False	False	
6	False	False	False	False	False	False	
7	False	False	False	False	False	False	
8	False	False	False	False	False	True	
9	False	False	False	False	False	True	

In [28]:

```
row_mask = data2.isin(values).any(axis=1)
data2[row_mask]
```

Out[28]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
0	1000004	P00128942	M	46-50	7	B	
1	1000009	P00113442	M	26-35	17	C	
2	1000010	P00288442	F	36-45	1	B	
3	1000010	P00145342	F	36-45	1	B	
4	1000011	P00053842	F	26-35	1	C	
...	...	...	...	...	...	...	...
233594	1006036	P00118942	F	26-35	15	B	
233595	1006036	P00254642	F	26-35	15	B	
233596	1006036	P00031842	F	26-35	15	B	
233597	1006037	P00124742	F	46-50	1	C	
233598	1006039	P00316642	F	46-50	0	B	

216224 rows × 12 columns



```
In [29]: data2_where=data2.where(data2 == 0)
(data2_where).head(10)
```


Out[29]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current_City_
0	NaN	NaN	NaN	NaN	NaN	NaN	
1	NaN	NaN	NaN	NaN	NaN	NaN	
2	NaN	NaN	NaN	NaN	NaN	NaN	
3	NaN	NaN	NaN	NaN	NaN	NaN	
4	NaN	NaN	NaN	NaN	NaN	NaN	
5	NaN	NaN	NaN	NaN	NaN	NaN	
6	NaN	NaN	NaN	NaN	NaN	NaN	
7	NaN	NaN	NaN	NaN	NaN	NaN	
8	NaN	NaN	NaN	NaN	NaN	NaN	
9	NaN	NaN	NaN	NaN	NaN	NaN	

In [30]: data2.query('(Product_Category_1 > Product_Category_2) & (Product_Category_2 > P

Out[30]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
46	1000090	P00117542	M	55+	13	C	
446	1000767	P00261542	M	26-35	12	C	
1026	1001667	P00020542	M	51-55	16	B	
1076	1001733	P00117542	M	18-25	14	B	
1152	1001837	P00185442	M	26-35	2	B	
...	...	...	...	...	...	...	...
232384	1004087	P00255842	M	0-17	4	C	
232442	1004204	P00326742	M	36-45	7	C	
232495	1004277	P00020542	M	36-45	16	A	
232805	1004795	P00255842	M	46-50	16	C	
233193	1005442	P00117542	M	26-35	7	C	

1089 rows × 12 columns

In [31]: *# Let's create a new dataframe*

```
food = pd.DataFrame({'Place':['Home', 'Home', 'Hotel', 'Hotel'],
                     'Time': ['Lunch', 'Dinner', 'Lunch', 'Dinner'],
                     'Food': ['Soup', 'Rice', 'Soup', 'Chapati'],
                     'Price($)': [10, 20, 30, 40]})
```

food

Out[31]:

	Place	Time	Food	Price(\$)
0	Home	Lunch	Soup	10
1	Home	Dinner	Rice	20
2	Hotel	Lunch	Soup	30
3	Hotel	Dinner	Chapati	40

In [32]: `food_indexed1=food.set_index('Place')`
 food_indexed1

Out[32]:

	Time	Food	Price(\$)
Place			
Home	Lunch	Soup	10
Home	Dinner	Rice	20
Hotel	Lunch	Soup	30
Hotel	Dinner	Chapati	40

In [33]: `food_indexed2=food.set_index(['Place', 'Time'])`
 food_indexed2

Out[33]:

		Food	Price(\$)
Place	Time		
Home	Lunch	Soup	10
	Dinner	Rice	20
Hotel	Lunch	Soup	30
	Dinner	Chapati	40

In [34]: `food_indexed2.reset_index()`

Out[34]:

	Place	Time	Food	Price(\$)
0	Home	Lunch	Soup	10
1	Home	Dinner	Rice	20
2	Hotel	Lunch	Soup	30
3	Hotel	Dinner	Chapati	40

In [35]:

```
sales=pd.DataFrame([['books','online', 200, 50],['books','retail', 250, 75],
                    ['toys','online', 100, 20],['toys','retail', 140, 30],
                    ['watches','online', 500, 100],['watches','retail', 600, 150],
                    ['computers','online', 1000, 200],['computers','retail', 1200, 300],
                    ['laptops','online', 1100, 400],['laptops','retail', 1400, 500],
                    ['smartphones','online', 600, 200],['smartphones','retail', 800, 250]],
                    columns=['Items', 'Mode', 'Price', 'Profit'])
sales
```

Out[35]:

	Items	Mode	Price	Profit
0	books	online	200	50
1	books	retail	250	75
2	toys	online	100	20
3	toys	retail	140	30
4	watches	online	500	100
5	watches	retail	600	150
6	computers	online	1000	200
7	computers	retail	1200	300
8	laptops	online	1100	400
9	laptops	retail	1400	500
10	smartphones	online	600	200
11	smartphones	retail	800	250

In [36]:

```
sales1=sales.set_index(['Items', 'Mode'])
sales1
```

Out[36]:

		Price	Profit
Items Mode			
books	online	200	50
	retail	250	75
toys	online	100	20
	retail	140	30
watches	online	500	100
	retail	600	150
computers	online	1000	200
	retail	1200	300
laptops	online	1100	400
	retail	1400	500
smartphones	online	600	200
	retail	800	250

```
In [37]: # View index
sales1.index
```

```
Out[37]: MultiIndex([(      'books', 'online'),
 (      'books', 'retail'),
 (      'toys', 'online'),
 (      'toys', 'retail'),
 (    'watches', 'online'),
 (    'watches', 'retail'),
 ( 'computers', 'online'),
 ( 'computers', 'retail'),
 (    'laptops', 'online'),
 (    'laptops', 'retail'),
 ('smartphones', 'online'),
 ('smartphones', 'retail')],
 names=['Items', 'Mode'])
```

```
In [38]: # Swap the column in multiple index
sales2=sales1.swaplevel('Mode', 'Items')
sales2
```

Out[38]:

		Price	Profit
Mode	Items		
online	books	200	50
retail	books	250	75
online	toys	100	20
retail	toys	140	30
online	watches	500	100
retail	watches	600	150
online	computers	1000	200
retail	computers	1200	300
online	laptops	1100	400
retail	laptops	1400	500
online	smartphones	600	200
retail	smartphones	800	250

```
In [39]: # sort the dataframe df2 by label  
data2.sort_index()
```

Out[39]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
0	1000004	P00128942	M	46-50	7	B	
1	1000009	P00113442	M	26-35	17	C	
2	1000010	P00288442	F	36-45	1	B	
3	1000010	P00145342	F	36-45	1	B	
4	1000011	P00053842	F	26-35	1	C	
...	...	...	...	...	...	...	...
233594	1006036	P00118942	F	26-35	15	B	
233595	1006036	P00254642	F	26-35	15	B	
233596	1006036	P00031842	F	26-35	15	B	
233597	1006037	P00124742	F	46-50	1	C	
233598	1006039	P00316642	F	46-50	0	B	

233599 rows × 12 columns



```
In [40]: data2.sort_values(by=['Product_Category_1'])
```

Out[40]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
233587	1006028	P00162442	M	18-25	4	A	
233586	1006026	P00110742	M	36-45	6	C	
33	1000059	P00177442	F	51-55	1	B	
40	1000073	P00112642	M	18-25	4	A	
42	1000080	P00112142	M	55+	1	C	
...	...	...	...	...	...	...	...
159738	1005916	P00286042	M	51-55	20	B	
159739	1005916	P00117542	M	51-55	20	B	
130715	1003435	P00220942	M	46-50	1	B	
46	1000090	P00117542	M	55+	13	C	
34	1000062	P00119242	F	36-45	3	A	

233599 rows × 12 columns



```
In [41]: data3 = data.copy()
data3.dtypes
```

```
Out[41]: User_ID          int64
Product_ID         object
Gender             object
Age               object
Occupation         int64
City_Category      object
Stay_In_Current_City_Years  object
Marital_Status     int64
Product_Category_1  int64
Product_Category_2  float64
Product_Category_3  float64
Purchase           float64
dtype: object
```

```
In [42]: data3['Gender'].describe()
```

```
Out[42]: count      233599
unique         2
top            M
freq          175772
Name: Gender, dtype: object
```

```
In [43]: data3['Age'].describe()
```

```
Out[43]: count      233599  
unique         7  
top           26-35  
freq         93428  
Name: Age, dtype: object
```

```
In [44]: data3['City_Category'].describe()
```

```
Out[44]: count      233599  
unique         3  
top            B  
freq         98566  
Name: City_Category, dtype: object
```

```
In [45]: data3['Gender'].unique()
```

```
Out[45]: array(['M', 'F'], dtype=object)
```

```
In [46]: data3['Age'].unique()
```

```
Out[46]: array(['46-50', '26-35', '36-45', '18-25', '51-55', '55+', '0-17'],  
              dtype=object)
```

```
In [47]: data3['Gender'].value_counts()
```

```
Out[47]: Gender  
M      175772  
F       57827  
Name: count, dtype: int64
```

```
In [48]: data3['City_Category'].value_counts()
```

```
Out[48]: City_Category  
B      98566  
C      72509  
A      62524  
Name: count, dtype: int64
```

```
In [49]: data3['Gender'].value_counts(ascending=True)
```

```
Out[49]: Gender  
F       57827  
M      175772  
Name: count, dtype: int64
```

```
In [50]: data3['City_Category'].value_counts(ascending=True)
```

```
Out[50]: City_Category  
A      62524  
C      72509  
B      98566  
Name: count, dtype: int64
```

```
In [51]: data4=data.copy()  
data4.max(0)
```



```
Out[51]: User_ID          1006040
Product_ID      P0099942
Gender          M
Age             55+
Occupation      20
City_Category   C
Stay_In_Current_City_Years  4+
Marital_Status  1
Product_Category_1  18
Product_Category_2  18.0
Product_Category_3  18.0
Purchase        188.0
dtype: object
```

```
In [55]: data4.describe()
```

```
Out[55]:
```

	User_ID	Occupation	Marital_Status	Product_Category_1	Product_Category_2
count	2.335990e+05	233599.000000	233599.000000	233599.000000	233599.000000
mean	1.003029e+06	8.085407	0.410070	5.276542	9.860000
std	1.726505e+03	6.521146	0.491847	3.736380	5.070000
min	1.000001e+06	0.000000	0.000000	1.000000	2.000000
25%	1.001527e+06	2.000000	0.000000	1.000000	5.000000
50%	1.003070e+06	7.000000	0.000000	5.000000	9.000000
75%	1.004477e+06	14.000000	1.000000	8.000000	15.000000
max	1.006040e+06	20.000000	1.000000	18.000000	18.000000

```
In [57]: data5 = data.copy()
```

```
In [58]: numeric_data = data5._get_numeric_data()
numeric_data.cov() # Covariance
```

```
Out[58]:
```

	User_ID	Occupation	Marital_Status	Product_Category_1	Product_Category_2
User_ID	2.980819e+06	-270.071348	16.883833	41.465376	41.465376
Occupation	-2.700713e+02	42.525351	0.070153	-0.254982	-0.254982
Marital_Status	1.688383e+01	0.070153	0.241914	0.034660	0.034660
Product_Category_1	4.146538e+01	-0.254982	0.034660	13.960536	13.960536
Product_Category_2	3.215471e+01	-0.022942	0.038662	6.358025	6.358025
Product_Category_3	5.139372e+00	-0.037906	0.015807	0.762418	0.762418
Purchase	-2.442961e+02	0.264979	-0.022546	0.086225	0.086225

```
In [59]: # view the correlation
data5.select_dtypes(include='number').corr()
```

Out[59]:

	User_ID	Occupation	Marital_Status	Product_Category_1	Product
User_ID	1.000000	-0.023988	0.019883	0.006428	
Occupation	-0.023988	1.000000	0.021872	-0.010465	
Marital_Status	0.019883	0.021872	1.000000	0.018860	
Product_Category_1	0.006428	-0.010465	0.018860	1.000000	
Product_Category_2	0.003669	-0.000693	0.015488	0.335272	
Product_Category_3	0.000722	-0.001411	0.007800	0.049523	
Purchase	-0.018074	0.005190	-0.005855	0.002948	

In [60]:

```
# view the top 25 rows of ranked dataframe
data5.select_dtypes(include='number').rank(axis=1).head(25)
```

Out[60]:

	User_ID	Occupation	Marital_Status	Product_Category_1	Product_Category_2	Prod
0	7.0	3.0	1.5	1.5	5.0	
1	7.0	6.0	1.0	2.0	3.0	
2	7.0	1.5	1.5	3.0	5.0	
3	7.0	1.5	1.5	3.0	4.0	
4	7.0	2.0	1.0	3.0	4.0	
5	7.0	1.5	1.5	3.0	4.0	
6	7.0	2.0	2.0	2.0	4.0	
7	7.0	1.5	1.5	3.0	4.0	
8	7.0	2.0	1.0	3.0	4.0	
9	7.0	4.0	1.0	2.0	3.0	
10	7.0	4.0	1.5	1.5	3.0	
11	7.0	4.0	1.0	3.0	2.0	
12	7.0	3.0	1.0	2.0	4.0	
13	7.0	3.0	1.0	2.0	4.0	
14	7.0	1.5	1.5	3.0	4.0	
15	7.0	3.0	1.0	2.0	5.0	
16	7.0	2.0	1.0	3.0	5.0	
17	7.0	2.0	1.0	3.0	4.0	
18	7.0	1.5	1.5	3.0	4.0	
19	7.0	1.5	1.5	3.0	4.0	
20	7.0	2.5	1.0	5.0	2.5	
21	7.0	4.5	1.0	2.0	3.0	
22	7.0	4.0	1.5	1.5	3.0	
23	7.0	5.0	1.5	1.5	3.0	
24	7.0	5.0	1.0	3.0	2.0	

In [61]: `data6=data.copy()
data6['Purchase'].aggregate(np.sum)`

C:\Windows\Temp\ipykernel_28304\339375412.py:2: FutureWarning: The provided callable <function sum at 0x000001E765EAFA60> is currently using Series.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

`data6['Purchase'].aggregate(np.sum)`

Out[61]: `np.float64(43828368.6)`

```
In [62]: data6['Purchase'].aggregate([np.sum, np.mean])
```

C:\Windows\Temp\ipykernel_28304\2741013109.py:1: FutureWarning: The provided callable <function sum at 0x000001E765EAF60> is currently using Series.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

```
data6['Purchase'].aggregate([np.sum, np.mean])
```

C:\Windows\Temp\ipykernel_28304\2741013109.py:1: FutureWarning: The provided callable <function mean at 0x000001E765ED4EA0> is currently using Series.mean. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "mean" instead.

```
data6['Purchase'].aggregate([np.sum, np.mean])
```

```
Out[62]: sum      4.382837e+07
         mean      1.876222e+02
         Name: Purchase, dtype: float64
```

```
In [63]: data6[['Product_Category_1', 'Product_Category_2', 'Product_Category_3']].agg
```

C:\Windows\Temp\ipykernel_28304\1314604894.py:1: FutureWarning: The provided callable <function mean at 0x000001E765ED4EA0> is currently using DataFrame.mean. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "mean" instead.

```
data6[['Product_Category_1', 'Product_Category_2', 'Product_Category_3']].agg
```

```
Out[63]: Product_Category_1    5.276542
         Product_Category_2    9.868283
         Product_Category_3   12.667379
         dtype: float64
```

```
In [64]: data6[['Product_Category_1', 'Product_Category_2', 'Product_Category_3']].agg
```

C:\Windows\Temp\ipykernel_28304\930593889.py:1: FutureWarning: The provided callable <function sum at 0x000001E765EAF60> is currently using Series.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

```
data6[['Product_Category_1', 'Product_Category_2', 'Product_Category_3']].agg
```

C:\Windows\Temp\ipykernel_28304\930593889.py:1: FutureWarning: The provided callable <function mean at 0x000001E765ED4EA0> is currently using Series.mean. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "mean" instead.

```
data6[['Product_Category_1', 'Product_Category_2', 'Product_Category_3']].agg
```

```
Out[64]:
```

	Product_Category_1	Product_Category_2	Product_Category_3
sum	1.232595e+06	2.305221e+06	2.959087e+06
mean	5.276542e+00	9.868283e+00	1.266738e+01

```
In [65]: data6.agg({'Product_Category_1' : np.sum , 'Product_Category_2' : np.mean})
```

C:\Windows\Temp\ipykernel_28304\246499583.py:1: FutureWarning: The provided callable <function sum at 0x000001E765EAF60> is currently using Series.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

```
data6.aggregate({'Product_Category_1' : np.sum , 'Product_Category_2' : np.mean})
```

C:\Windows\Temp\ipykernel_28304\246499583.py:1: FutureWarning: The provided callable <function mean at 0x000001E765ED4EA0> is currently using Series.mean. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "mean" instead.

```
data6.aggregate({'Product_Category_1' : np.sum , 'Product_Category_2' : np.mean})
```

```
Out[65]: Product_Category_1    1.232595e+06
        Product_Category_2    9.868283e+00
        dtype: float64
```

```
In [66]: data8=data.copy()
        data8.groupby('Gender')
```

```
Out[66]: <pandas.core.groupby.generic.DataFrameGroupBy object at 0x000001E77A7F4440>
```

```
In [68]: # view groups of Gender column
        data8.groupby('Gender').groups
```

```
Out[68]: {'F': [2, 3, 4, 14, 18, 23, 24, 25, 33, 34, 39, 41, 43, 44, 47, 48, 50, 51, 52,
75, 76, 77, 83, 84, 108, 109, 110, 111, 113, 114, 115, 119, 147, 149, 155, 156,
157, 160, 163, 170, 184, 185, 186, 187, 188, 189, 190, 191, 207, 209, 211, 212,
223, 224, 225, 235, 245, 246, 247, 249, 250, 281, 286, 294, 295, 296, 297, 298,
299, 310, 312, 320, 321, 322, 323, 330, 334, 335, 336, 337, 339, 340, 341, 351,
353, 358, 364, 365, 371, 372, 380, 381, 382, 383, 384, 385, 386, 387, 388, 410,
...], 'M': [0, 1, 5, 6, 7, 8, 9, 10, 11, 12, 13, 15, 16, 17, 19, 20, 21, 22, 2
6, 27, 28, 29, 30, 31, 32, 35, 36, 37, 38, 40, 42, 45, 46, 49, 53, 54, 55, 56,
57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 78, 79,
80, 81, 82, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 10
1, 102, 103, 104, 105, 106, 107, 112, 116, 117, 118, 120, 121, 122, 123, 124, 1
25, 126, 127, 128, 129, 130, 131, ...]}
```

```
In [69]: # apply aggregation function sum with groupby
        data8.groupby('Gender').sum()
```

Out[69]:

	User_ID	Product_ID	Age	Occ
	Gender			
F	58008700738	P00288442P00145342P00053842P00159542P00265242P...	36-	
			4536-	
			4526-	
			3526-	
			3518-	
			2546-	
			5046-	
			5046-	
			5051-	
			553...	
M	176297953995	P00128942P00113442P00350442P00155442P0094542P0...	46-	
			5026-	
			3546-	
			5046-	
			5046-	
			5026-	
			3518-	
			2526-	
			3526-	
			352...	

In [70]:

```
# alternative way to apply aggregation function sum
data8.groupby('Gender').agg(np.sum)
```

C:\Windows\Temp\ipykernel_28304\132593672.py:2: FutureWarning: The provided callable <function sum at 0x000001E765EAF60> is currently using DataFrameGroupBy.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.
 data8.groupby('Gender').agg(np.sum)

Out[70]:

	User_ID	Product_ID	Age	Occ
Gender				
F	58008700738	P00288442P00145342P00053842P00159542P00265242P...	36-4536-4526-3526-3518-2546-5046-5046-5051-553...	
M	176297953995	P00128942P00113442P00350442P00155442P0094542P0...	46-5026-3546-5046-5046-5026-3518-2526-3526-352...	

```
In [71]: # attribute access in python pandas
data8_grouped = data8.groupby('Gender')
print(data8_grouped.agg(np.size))
```

	User_ID	Product_ID	Age	Occupation	City_Category	\
Gender						
F	57827	57827	57827	57827	57827	
M	175772	175772	175772	175772	175772	
Stay_In_Current_City_Years						
Marital_Status						
Product_Category_1						
Product_Category_2						
Product_Category_3						
Purchase						
F		57827		57827	57827	
M		175772		175772	175772	

```
In [72]: data8.groupby('Gender')['Purchase'].agg([np.sum, np.mean])
```

C:\Windows\Temp\ipykernel_28304\1762637521.py:1: FutureWarning: The provided callable <function sum at 0x000001E765EAF60> is currently using SeriesGroupBy.sum. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "sum" instead.

```
data8.groupby('Gender')['Purchase'].agg([np.sum, np.mean])
```

C:\Windows\Temp\ipykernel_28304\1762637521.py:1: FutureWarning: The provided callable <function mean at 0x000001E765ED4EA0> is currently using SeriesGroupBy.mean. In a future version of pandas, the provided callable will be used directly. To keep current behavior pass the string "mean" instead.

```
data8.groupby('Gender')['Purchase'].agg([np.sum, np.mean])
```

Out[72]:

	sum	mean
Gender		
F	10845735.0	187.554862
M	32982633.6	187.644412

```
In [74]: data9=data.copy()
score = lambda x: (x - x.mean()) / x.std()*10
print(data9.groupby('Gender')['Purchase'].transform(score).head(5))
```

```
0    -234.707627
1    -228.101519
2    -201.325095
3    -200.158364
4    -196.658171
Name: Purchase, dtype: float64
```

```
In [76]: data10=data.copy()
data10.groupby('Gender').filter(lambda x: len(x) > 4)
```

Out[76]:

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Stay_In_Current
0	1000004	P00128942	M	46-50	7	B	
1	1000009	P00113442	M	26-35	17	C	
2	1000010	P00288442	F	36-45	1	B	
3	1000010	P00145342	F	36-45	1	B	
4	1000011	P00053842	F	26-35	1	C	
...	...	...	...	...	...	...	...
233594	1006036	P00118942	F	26-35	15	B	
233595	1006036	P00254642	F	26-35	15	B	
233596	1006036	P00031842	F	26-35	15	B	
233597	1006037	P00124742	F	46-50	1	C	
233598	1006039	P00316642	F	46-50	0	B	

233599 rows × 12 columns



```
In [77]: # Let's create two dataframes
```



```

batsmen = pd.DataFrame({
    'id':[1,2,3,4,5],
    'Name': ['Rohit', 'Dhawan', 'Virat', 'Dhoni', 'Kedar'],
    'subject_id':['sub1','sub2','sub4','sub6','sub5']})

bowler = pd.DataFrame(
    {'id':[1,2,3,4,5],
    'Name': ['Kumar', 'Bumrah', 'Shami', 'Kuldeep', 'Chahal'],
    'subject_id':['sub2','sub4','sub3','sub6','sub5']})
print(batsmen)
print(bowler)

```

```

   id  Name subject_id
0    1  Rohit      sub1
1    2 Dhawan      sub2
2    3  Virat      sub4
3    4  Dhoni      sub6
4    5  Kedar      sub5
   id  Name subject_id
0    1  Kumar      sub2
1    2  Bumrah     sub4
2    3   Shami     sub3
3    4 Kuldeep     sub6
4    5  Chahal     sub5

```

```

In [78]: # merge two dataframes on a key
pd.merge(batsmen, bowler, on='id')

```

```

Out[78]:
```

	id	Name_x	subject_id_x	Name_y	subject_id_y
0	1	Rohit	sub1	Kumar	sub2
1	2	Dhawan	sub2	Bumrah	sub4
2	3	Virat	sub4	Shami	sub3
3	4	Dhoni	sub6	Kuldeep	sub6
4	5	Kedar	sub5	Chahal	sub5

```

In [79]: # merge two dataframes on multiple keys
pd.merge(batsmen, bowler, on=['id', 'subject_id'])

```

```

Out[79]:
```

	id	Name_x	subject_id	Name_y
0	4	Dhoni	sub6	Kuldeep
1	5	Kedar	sub5	Chahal

```

In [80]: # Left join
pd.merge(batsmen, bowler, on='subject_id', how='left')

```

Out[80]:

	id_x	Name_x	subject_id	id_y	Name_y
0	1	Rohit	sub1	NaN	NaN
1	2	Dhawan	sub2	1.0	Kumar
2	3	Virat	sub4	2.0	Bumrah
3	4	Dhoni	sub6	4.0	Kuldeep
4	5	Kedar	sub5	5.0	Chahal

In [81]: *# right join*
 pd.merge(batsmen, bowler, on='subject_id', how='right')

Out[81]:

	id_x	Name_x	subject_id	id_y	Name_y
0	2.0	Dhawan	sub2	1	Kumar
1	3.0	Virat	sub4	2	Bumrah
2	NaN	NaN	sub3	3	Shami
3	4.0	Dhoni	sub6	4	Kuldeep
4	5.0	Kedar	sub5	5	Chahal

In [82]: *# outer join*
 pd.merge(batsmen, bowler, on='subject_id', how='outer')

Out[82]:

	id_x	Name_x	subject_id	id_y	Name_y
0	1.0	Rohit	sub1	NaN	NaN
1	2.0	Dhawan	sub2	1.0	Kumar
2	NaN	NaN	sub3	3.0	Shami
3	3.0	Virat	sub4	2.0	Bumrah
4	5.0	Kedar	sub5	5.0	Chahal
5	4.0	Dhoni	sub6	4.0	Kuldeep

In [83]: *# inner join*
 pd.merge(batsmen, bowler, on='subject_id', how='inner')

Out[83]:

	id_x	Name_x	subject_id	id_y	Name_y
0	2	Dhawan	sub2	1	Kumar
1	3	Virat	sub4	2	Bumrah
2	4	Dhoni	sub6	4	Kuldeep
3	5	Kedar	sub5	5	Chahal

In [84]: *# Let's create two dataframes*
 batsmen = pd.DataFrame({

```

'id':[1,2,3,4,5],
'Name': ['Rohit', 'Dhawan', 'Virat', 'Dhoni', 'Kedar'],
'subject_id':['sub1','sub2','sub4','sub6','sub5']})

bowler = pd.DataFrame(
{'id':[1,2,3,4,5],
'Name': ['Kumar', 'Bumrah', 'Shami', 'Kuldeep', 'Chahal'],
'subject_id':['sub2','sub4','sub3','sub6','sub5']})

print(batsmen)

print(bowler)

```

```

id    Name subject_id
0    1  Rohit      sub1
1    2 Dhawan      sub2
2    3  Virat      sub4
3    4  Dhoni      sub6
4    5  Kedar      sub5
id    Name subject_id
0    1   Kumar      sub2
1    2  Bumrah      sub4
2    3   Shami      sub3
3    4 Kuldeep      sub6
4    5  Chahal      sub5

```

```

In [85]: # concatenate the dataframes
team=[batsmen, bowler]
pd.concat(team)

```

```

Out[85]:
   id  Name subject_id
0    1  Rohit      sub1
1    2 Dhawan      sub2
2    3  Virat      sub4
3    4  Dhoni      sub6
4    5  Kedar      sub5
0    1   Kumar      sub2
1    2  Bumrah      sub4
2    3   Shami      sub3
3    4 Kuldeep      sub6
4    5  Chahal      sub5

```

```

In [86]: # associate keys with the dataframes
pd.concat(team, keys=['x', 'y'])

```

Out[86]:

		id	Name	subject_id
x	0	1	Rohit	sub1
	1	2	Dhawan	sub2
	2	3	Virat	sub4
	3	4	Dhoni	sub6
	4	5	Kedar	sub5
y	0	1	Kumar	sub2
	1	2	Bumrah	sub4
	2	3	Shami	sub3
	3	4	Kuldeep	sub6
	4	5	Chahal	sub5

In [87]: `pd.concat(team, keys=['x', 'y'], ignore_index=True)`

Out[87]:

	id	Name	subject_id
0	1	Rohit	sub1
1	2	Dhawan	sub2
2	3	Virat	sub4
3	4	Dhoni	sub6
4	5	Kedar	sub5
5	1	Kumar	sub2
6	2	Bumrah	sub4
7	3	Shami	sub3
8	4	Kuldeep	sub6
9	5	Chahal	sub5

In [88]: `pd.concat(team, axis=1)`

Out[88]:

	id	Name	subject_id	id	Name	subject_id
0	1	Rohit	sub1	1	Kumar	sub2
1	2	Dhawan	sub2	2	Bumrah	sub4
2	3	Virat	sub4	3	Shami	sub3
3	4	Dhoni	sub6	4	Kuldeep	sub6
4	5	Kedar	sub5	5	Chahal	sub5

```
In [91]: import pandas as pd
result = pd.concat([batsmen, bowler], ignore_index=True)
```

```
In [92]: result = pd.concat([batsmen, bowler], axis=0)
```

```
In [96]: data11=data.copy()

data11.columns
```

```
Out[96]: Index(['User_ID', 'Product_ID', 'Gender', 'Age', 'Occupation', 'City_Category',
              'Stay_In_Current_City_Years', 'Marital_Status', 'Product_Category_1',
              'Product_Category_2', 'Product_Category_3', 'Purchase'],
              dtype='object')
```

```
In [98]: data12=(pd.melt(frame=data11, id_vars=['User_ID', 'Product_ID', 'Gender', 'Age', 'City_Category', 'Marital_Status', 'Purchase'],
                        value_vars=['Product_Category_1', 'Product_Category_2', 'Product_Category_3'],
                        var_name='Product_Category', value_name='Amount'))
data12.head(10)
```

```
Out[98]:
```

	User_ID	Product_ID	Gender	Age	Occupation	City_Category	Marital_Status	Purchase
--	---------	------------	--------	-----	------------	---------------	----------------	----------

0	1000004	P00128942	M	46-50	7	B	1
---	---------	-----------	---	-------	---	---	---

1	1000009	P00113442	M	26-35	17	C	0
---	---------	-----------	---	-------	----	---	---

2	1000010	P00288442	F	36-45	1	B	1
---	---------	-----------	---	-------	---	---	---

3	1000010	P00145342	F	36-45	1	B	1
---	---------	-----------	---	-------	---	---	---

4	1000011	P00053842	F	26-35	1	C	0
---	---------	-----------	---	-------	---	---	---

5	1000013	P00350442	M	46-50	1	C	1
---	---------	-----------	---	-------	---	---	---

6	1000013	P00155442	M	46-50	1	C	1
---	---------	-----------	---	-------	---	---	---

7	1000013	P0094542	M	46-50	1	C	1
---	---------	----------	---	-------	---	---	---

8	1000015	P00161842	M	26-35	7	A	0
---	---------	-----------	---	-------	---	---	---

9	1000022	P00067942	M	18-25	15	A	0
---	---------	-----------	---	-------	----	---	---



In [100...

```
data13 = data12[['Product_Category', 'Amount']]

# create an index column
data13 = data13.reset_index()

data14 = data13.pivot(index='index', columns='Product_Category', values='Amount')
data14.head(25)
```

Out[100...

	Product_Category	Product_Category_1	Product_Category_2	Product_Category_3
	index			
	0	1.0	NaN	NaN
	1	3.0	NaN	NaN
	2	5.0	NaN	NaN
	3	4.0	NaN	NaN
	4	4.0	NaN	NaN
	5	2.0	NaN	NaN
	6	1.0	NaN	NaN
	7	2.0	NaN	NaN
	8	10.0	NaN	NaN
	9	5.0	NaN	NaN
	10	1.0	NaN	NaN
	11	5.0	NaN	NaN
	12	5.0	NaN	NaN
	13	5.0	NaN	NaN
	14	10.0	NaN	NaN
	15	2.0	NaN	NaN
	16	15.0	NaN	NaN
	17	5.0	NaN	NaN
	18	5.0	NaN	NaN
	19	2.0	NaN	NaN
	20	5.0	NaN	NaN
	21	1.0	NaN	NaN
	22	1.0	NaN	NaN
	23	1.0	NaN	NaN
	24	5.0	NaN	NaN

In [101...

```
cols=pd.MultiIndex.from_tuples([('weight', 'kg'), ('weight', 'pounds')])
```

```
df15=pd.DataFrame([[75,165], [60, 132]],
                  index=['husband', 'wife'],
                  columns=cols)
df15
```

Out[101...

weight		
	kg	pounds
husband	75	165
wife	60	132

In [102...
df16=df15.stack()
df16

C:\Windows\Temp\ipykernel_28304\1171163659.py:1: FutureWarning: The previous implementation of stack is deprecated and will be removed in a future version of pandas. See the What's New notes for pandas 2.1.0 for details. Specify future_stack=True to adopt the new implementation and silence this warning.

```
df16=df15.stack()
```

Out[102...

weight		
husband	kg	75
	pounds	165
wife	kg	60
	pounds	132

In [103...
df16.unstack()

Out[103...

weight		
	kg	pounds
husband	75	165
wife	60	132

In [104...
display maximum rows
pd.get_option("display.max_rows")

Out[104... 60

In [105...
display maximum columns
pd.get_option("display.max_columns")

Out[105... 20

In [106...
set maximum rows
pd.set_option("display.max_rows", 80)
pd.get_option("display.max_rows")

Out[106... 80

```
In [107... # set maximum columns
pd.set_option("display.max_columns", 30)
pd.get_option("display.max_columns")
```

```
Out[107... 30
```

```
In [108... # display maximum rows
pd.reset_option("display.max_rows")
pd.get_option("display.max_rows")
```

```
Out[108... 60
```

```
In [109... # display maximum columns
pd.reset_option("display.max_columns")
pd.get_option("display.max_columns")
```

```
Out[109... 20
```

```
In [110... # description of the display maximum rows parameter
pd.describe_option("display.max_rows")
```

```
display.max_rows : int
    If max_rows is exceeded, switch to truncate view. Depending on
    `large_repr`, objects are either centrally truncated or printed as
    a summary view. 'None' value means unlimited.

    In case python/IPython is running in a terminal and `large_repr`
    equals 'truncate' this can be set to 0 and pandas will auto-detect
    the height of the terminal and print a truncated object which fits
    the screen height. The IPython notebook, IPython qtconsole, or
    IDLE do not run in a terminal and hence it is not possible to do
    correct auto-detection.
    [default: 60] [currently: 60]
```

```
In [111... # set the parameter value with option_context
with pd.option_context("display.max_rows",10):
    print(pd.get_option("display.max_rows"))
    print(pd.get_option("display.max_rows"))
```

```
10
10
```

30. Summary and conclusion

In this kernel, I have explored pandas and important data analysis tools of pandas.

I have used the Black Friday dataset and explore various functionalities offered by pandas.

I have shed light on important functionalities of pandas like aggregations in pandas, iteration in pandas, Pandas GroupBy operations, Pandas merging and joining.

I have also discussed Pandas concatenation operation, Reshaping by melt and pivot and Reshaping by stacking and unstacking.

I have also discussed basic functionality in Pandas, descriptive statistics in Pandas and statistical functions in Pandas.

Lastly, I have discussed options and customization options with Pandas.

In []: